

Load



Interaktive Medien 3

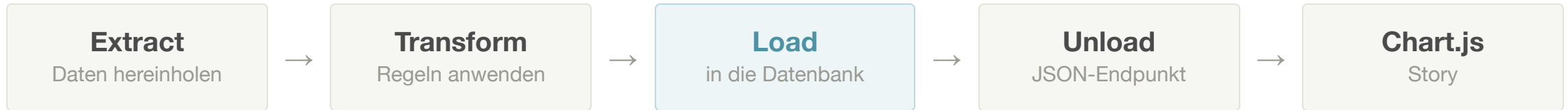
nick.schneeberger@fhgr.ch

Inhalt

- Was eine Datenbank ist
- Das Datenmodell: von der Zeile zur Tabelle
- SQL: die Sprache der Datenbank
- PDO: die Brücke von PHP zur Datenbank
- Was beim zweiten Aufruf passiert

Danach: das eigene Datenmodell auf Papier und ein Code-Along.

Wo wir stehen



- **Extract** hat beantwortet: Woher kommen die Daten, und wie lesen wir sie?
- **Transform** hat beantwortet: Welche Daten brauchen wir für unsere Frage, was bedeuten sie, und in welche Form bringen wir sie?
- **Load** beantwortet: Wie kommen sie an einen Ort, an dem sie bleiben?

Bisher leben die 258 Datensätze nur, solange das Skript läuft.

Wir bauen heute den Ort und den Weg dorthin.

Extract → Transform → Load → Datenbank → Unload → Chart.js

Was ist eine Datenbank?

Warum nicht einfach eine Datei?

Eine JSON-Datei zu speichern wäre möglich.

- Für 258 Zeilen funktioniert das sogar gut
- Ihr könnt sie öffnen und lesen
- Sie liegt einfach im Ordner

Das ist mühsam:

- Eine einzelne Stadt heraussuchen müsst ihr selbst programmieren
- Eine Zeile ergänzen heisst: die ganze Datei neu schreiben
- Zwei Abrufe gleichzeitig können die Datei zerstören
- Sortieren und Zählen macht niemand für euch

Eine Datenbank ist ein Programm, das genau diese Arbeit übernimmt.

Sie beantwortet Fragen an die Daten, statt nur Text aufzubewahren.

Eine Tabelle ist eine Liste mit festen Spalten

TABELLE HEAT_SUMMERS

id	city	year	measurement_days	hot_days	max_temperature_c
1	Bern	2022	92	14	34.6
2	Bern	2023	92	12	36.3
3	Chur	2023	92	14	35.4

- Eine **Zeile** ist ein Datensatz
- Eine **Spalte** ist ein Feld dieses Datensatzes
- Alle Zeilen haben dieselben Spalten
- Eine **Datenbank** enthält mehrere Tabellen
- Jede Spalte hat einen festen Datentyp
- Diese Struktur heisst **Schema**

Relational

Daten stehen in Tabellen und sind untereinander verknüpft.

- Die Spalten stehen vorher fest
- Abgefragt wird mit SQL
- MySQL, MariaDB, PostgreSQL, SQLite

Passt, wenn alle Datensätze gleich gebaut sind.

Nicht relational

Daten stehen als einzelne Dokumente nebeneinander.

- Jedes Dokument darf andere Felder haben
- Jedes System hat seine eigene Abfragesprache
- MongoDB, Firebase, Redis

Passt, wenn die Struktur offen bleiben muss.

Womit wir arbeiten

Jetzt: auf eurem Rechner

- MAMP startet eine MySQL-Datenbank auf eurem Laptop
- PHP startet ihr weiterhin mit `php -S localhost:8000`
- phpMyAdmin liegt auf der MAMP-Startseite

Am Schluss: auf einem Webhosting

- Dort läuft meist MariaDB, eine nahe Verwandte von MySQL
- Datenbank und Zugangsdaten kommen aus dem Hosting-Panel
- Bedient wird sie ebenfalls über phpMyAdmin

Beide sprechen dieselbe Sprache, deshalb läuft derselbe Code an beiden Orten.

Unterschiedlich sind nur die Zugangsdaten, und die stehen in einer eigenen Datei.

Extract → Transform → Load → Datenbank → Unload → Chart.js

Das Datenmodell

Von der Zeile im Resultat zur Tabelle in der Datenbank

Eine Zeile bleibt eine Zeile

RESULTAT AUS DEM TRANSFORM

```
{
  "city": "Bern",
  "year": 2023,
  "measurement_days": 92,
  "hot_days": 12,
  "max_temperature_c": 36.3
}
```



ZEILE IN DER DATENBANK

Spalte	Wert
city	Bern
year	2023
measurement_days	92
hot_days	12
max_temperature_c	36.3

Die Untersuchungseinheit aus dem Transform wird zur Zeile in der Tabelle.

Bei uns ist das eine Stadt in einem Sommer.

Jede Spalte bekommt einen Datentyp

Typ	Wofür	Bei uns
INT	ganze Zahlen	id
SMALLINT	kleine ganze Zahlen	year, hot_days
DECIMAL(4,1)	Kommazahl mit fester Genauigkeit	max_temperature_c
VARCHAR(80)	Text bis zu einer Höchstlänge	city
DATE, DATETIME	Datum und Zeitpunkt	in vielen Projekten
BOOLEAN	ja oder nein	in vielen Projekten

Der Typ ist eine Zusage: Was nicht hineinpasst, bleibt draussen.

Der Primärschlüssel

Jede Zeile braucht eine eindeutige Nummer, an der man genau sie erkennt.

```
id INT AUTO_INCREMENT PRIMARY KEY
```

- **PRIMARY KEY** heisst: Dieser Wert kommt kein zweites Mal vor
- **AUTO_INCREMENT** heisst: Die Datenbank vergibt die Nummer selbst
- Ihr schreibt diese Spalte beim Einfügen deshalb nie mit

Ohne Primärschlüssel könnt ihr eine einzelne Zeile später nicht sicher ansprechen.

Die `id` in unseren Zeilen

DREI ZEILEN AUS HEAT_SUMMERS

id	city	year	hot_days
1	Bern	2022	14
2	Bern	2023	12
3	Chur	2023	14

- Das Jahr 2023 steht zweimal da, die `id` nie
- Stadt und Jahr zusammen wären auch eindeutig, aber umständlich
- Mit `WHERE id = 2` erwischt ihr genau eine Zeile

Die Nummer bedeutet nichts, sie zeigt nur auf genau diese Zeile.

«Bern» steht 86-mal da

ALLES IN EINER TABELLE

city	year	hot_days
Bern	2021	0
Bern	2022	14
Bern	2023	12
...	...	...

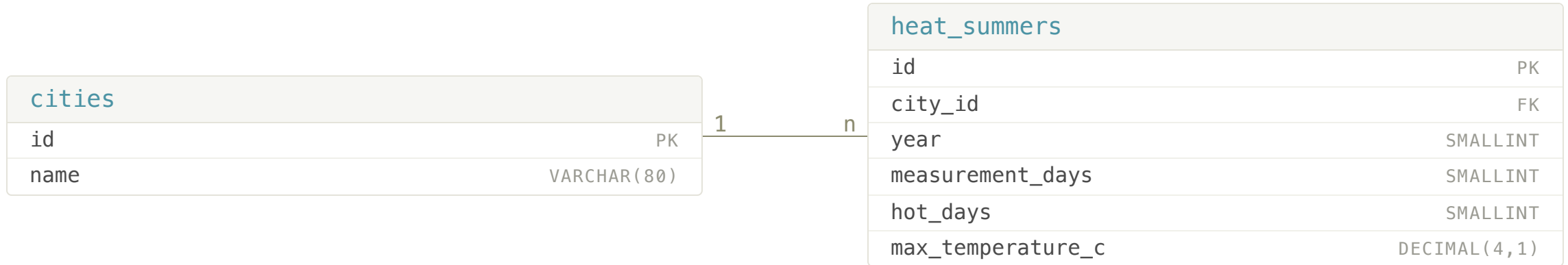
Drei Städte, 86 Jahre, 258 Zeilen.

- Jeder Stadtname steht 86-mal in der Tabelle
- Ein Tippfehler erzeugt eine vierte Stadt, die es nicht gibt
- Eine Umbenennung müsstet ihr 86-mal machen

Wiederholte Werte sind hier kein Platzproblem, sondern ein Fehlerrisiko.

Bei sehr grossen Datenmengen werden sie irgendwann auch ein Platzproblem.

Darum: Zwei Tabellen und ein Fremdschlüssel



- Der Stadtname steht jetzt genau einmal in **cities**
- **city_id** verweist auf die **id** in **cities** und heisst deshalb **Fremdschlüssel**
- Gelesen wird die Linie als: eine Stadt hat viele Sommer
- Jeder Sommer gehört aber nur zu einer Stadt

So plant ihr euer eigenes Modell

Vier Fragen, und zwar auf Papier und vor dem ersten SQL-Befehl.

- Was bedeutet eine Zeile in euren Daten?
- Welche Felder hat sie, und welchen Datentyp hat jedes davon?
- Welcher Wert wiederholt sich in fast jeder Zeile?
- Woran erkennt ihr eine einzelne Zeile eindeutig?

Der wiederholte Wert ist der Kandidat für die zweite Tabelle.

Mehr als zwei Tabellen braucht in diesem Kurs kein Projekt – ausser ihr kombiniert mehrere Datensätze.

Wann eine Tabelle genügt

Eine Tabelle

- Kein Wert wiederholt sich auffällig
- Alle Felder beschreiben dieselbe Sache
- Beispiel: Messwerte einer einzigen Station

Zwei Tabellen

- Ein Name oder eine Kategorie wiederholt sich ständig
- Diese Sache hat eigene Felder wie Koordinaten oder Farbe
- Beispiel: Städte und ihre Messungen

Eine schlecht geschnittene zweite Tabelle kostet mehr, als sie einbringt.

Im Zweifel klein anfangen und die Aufteilung begründen können.

Extract → Transform → Load → Datenbank → Unload → Chart.js

SQL

Die Sprache, in der man mit einer Datenbank redet

Vier Befehle reichen für alles

Befehl	Was er tut	Wo wir ihn brauchen
INSERT	schreibt eine Zeile hinein	heute, in <code>load.php</code>
SELECT	holt Zeilen heraus	heute zum Kontrollieren, später in <code>unload.php</code>
UPDATE	ändert vorhandene Zeilen	selten
DELETE	löscht Zeilen	zum Aufräumen

SQL ist eine eigene Sprache und hat mit PHP nichts zu tun.

Wir schicken sie als Text an die Datenbank und bekommen ein Resultat zurück.

Die Tabellen anlegen

```
CREATE TABLE cities (  
  id    INT AUTO_INCREMENT PRIMARY KEY,  
  name  VARCHAR(80) NOT NULL UNIQUE  
);  
  
CREATE TABLE heat_summers (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  city_id     INT NOT NULL,  
  year        SMALLINT NOT NULL,  
  measurement_days SMALLINT NOT NULL,  
  hot_days     SMALLINT NOT NULL,  
  max_temperature_c DECIMAL(4,1),  
  FOREIGN KEY (city_id) REFERENCES cities(id)  
);
```

- `NOT NULL` verlangt einen Wert, `UNIQUE` verbietet Wiederholungen
- `max_temperature_c` darf leer bleiben und ist dann `NULL`
- Dasselbe könnt ihr in phpMyAdmin auch zusammenklicken

INSERT schreibt eine Zeile

```
INSERT INTO cities (name)
VALUES ('Bern');
```

```
INSERT INTO heat_summers (city_id, year, measurement_days, hot_days, max_temperature_c)
VALUES (1, 2023, 92, 12, 36.3);
```

- Zuerst die Spalten, dann in derselben Reihenfolge die Werte
- Text steht in einfachen Anführungszeichen, Zahlen stehen ohne
- Die Spalte `id` fehlt, weil die Datenbank sie selbst vergibt

258-mal dieser Befehl mit anderen Werten – das ist der ganze Ladevorgang.

SELECT zum Nachschauen

```
SELECT * FROM heat_summers;
```

```
SELECT year, hot_days  
FROM heat_summers  
WHERE city_id = 1  
ORDER BY year;
```

- Der Stern holt alle Spalten
- **WHERE** wählt Zeilen aus
- **ORDER BY** sortiert das Resultat

Aus diesem Befehl wird später der JSON-Endpunkt für das Frontend.

Heute braucht ihr das nur, um zu prüfen, ob das Laden geklappt hat.

UPDATE und DELETE

```
UPDATE heat_summers SET hot_days = 13 WHERE id = 42;
```

```
DELETE FROM heat_summers WHERE id = 42;
```

```
DELETE FROM heat_summers;
```

- Beide wirken auf alle Zeilen, auf die die Bedingung zutrifft
- Ohne `WHERE` trifft sie auf jede Zeile zu
- Die dritte Zeile leert also die ganze Tabelle

Es gibt keine Rückfrage und kein Rückgängig.

phpMyAdmin

phpMyAdmin ist eine Oberfläche im Browser für eure Datenbank.

- Tabellen anlegen, Daten ansehen, Zeilen löschen
- Jede Aktion zeigt euch das SQL dazu an
- Der Reiter «SQL» nimmt eigene Befehle entgegen

Was ihr klickt, könnt ihr also auch lesen und lernen.

Angelegt wird die Tabelle einmal von Hand.

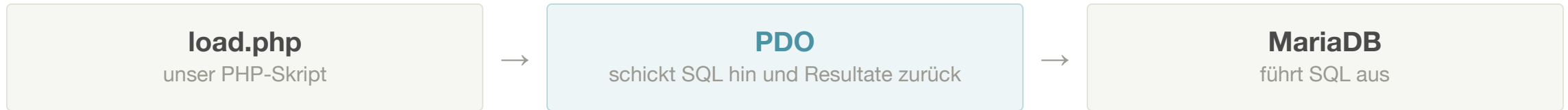
Gefüllt wird sie danach nur noch von `load.php`.

Extract → Transform → Load → Datenbank → Unload → Chart.js

PDO

Wie PHP mit der Datenbank spricht

PHP spricht kein SQL



- PDO ist fest in PHP eingebaut und heisst PHP Data Objects
- Wir übergeben SQL als Text und bekommen PHP-Arrays zurück
- Für MySQL, MariaDB, PostgreSQL und SQLite funktioniert es gleich

PDO ist ein Helfer wie `fetchJson()` beim Extract, nur für Datenbanken statt APIs.

Die Verbindung aufbauen

CONFIG.PHP – EINMAL IM HAUPTORDNER

```
<?php
$host      = '127.0.0.1';
$dbname    = 'im3_hitzesommer';
$username  = 'root';
$password  = 'root';

$dsn = "mysql:host=$host;port=8889;dbname=$dbname;charset=utf8mb4";
```

LOAD.PHP – IRGENDWO IM KURSORDNER

```
// drei Ordner nach oben zum Hauptordner
require __DIR__ . '/../../../config.php';

$pdo = new PDO($dsn, $username, $password, $options);
```

- Es gibt genau eine `config.php` für alle Übungen
- Der DSN sagt PDO, welche Datenbank wo liegt
- Der Host heisst `127.0.0.1` und nicht `localhost`

config.php enthält euer Passwort und darf nie auf GitHub landen.

Tragt die Datei in `.gitignore` ein, bevor ihr das erste Mal committet.

Zugangsdaten gehören nicht ins Repository

So arbeitet das Team damit.

- `config.php` steht in `.gitignore`
- Eine `config.template.php` ohne Werte liegt im Repository
- Jede Person kopiert sie und trägt ihre eigenen Werte ein

.GITIGNORE

`config.php`

Auch KI-Werkzeuge bekommen diese Datei nicht zu sehen.

Ein Passwort, das einmal in einem Commit steht, bleibt in der Historie stehen.

Wer es trotzdem hochgeladen hat, ändert sofort das Passwort.

Erst vorbereiten, dann ausführen

Werte schreiben wir nie direkt in den SQL-Text.

```
$insertSummer = $pdo->prepare(  
    'INSERT INTO heat_summers (city_id, year, measurement_days, hot_days, max_temperature_c)  
    VALUES (:city_id, :year, :measurement_days, :hot_days, :max_temperature_c)'  
);  
  
$insertSummer->execute([  
    'city_id' => 1,  
    'year' => 2023,  
    'measurement_days' => 92,  
    'hot_days' => 12,  
    'max_temperature_c' => 36.3,  
]);
```

- Die Namen mit Doppelpunkt sind Platzhalter und keine Werte
- `prepare` schickt den Bauplan, `execute` schickt die Werte
- Die Datenbank hält beides getrennt, deshalb kann kein Wert zu Code werden

Einmal vorbereiten, 258-mal ausführen

```
foreach ($rows as $row) {  
    $insertSummer->execute([  
        'city_id'           => $cityIds[$row['city']],  
        'year'              => $row['year'],  
        'measurement_days' => $row['measurement_days'],  
        'hot_days'          => $row['hot_days'],  
        'max_temperature_c' => $row['max_temperature_c'],  
    ]);  
}
```

- `prepare` steht vor der Schleife und läuft genau einmal
- `execute` steht in der Schleife und läuft für jede Zeile
- Das ist schneller und lesbarer als 258 zusammengebaute Befehle

Die Werte kommen unverändert aus dem Transform.

Woher kommt die `city_id`?

Im Transform steht «Bern», in der Tabelle braucht es die Nummer von Bern.

```
$findCity->execute([$city]);  
$id = $findCity->fetchColumn();  
  
if ($id === false) {  
    $insertCity->execute([$city]);  
    $id = $pdo->lastInsertId();  
}
```

- `fetchColumn` holt den ersten Wert der ersten gefundenen Zeile
- Ohne Treffer liefert es `false`, und dann legen wir die Stadt an
- `lastInsertId` gibt die Nummer zurück, die die Datenbank gerade vergeben hat

Was beim zweiten Aufruf passiert

NACH DEM ERSTEN AUFRUF

258 Zeilen



NACH DEM ZWEITEN AUFRUF

516 Zeilen

- `INSERT` fragt nicht, ob es die Zeile schon gibt
- Beim Entwickeln ruft ihr `load.php` zwanzigmal auf
- Der Chart zeigt danach jeden Sommer doppelt

Ein Ladeskript muss man mehrmals aufrufen können.

Zwei Muster, je nach Datenquelle

Stand neu schreiben

Für einen historischen Datensatz, der jedes Mal vollständig kommt.

```
$pdo->exec('DELETE FROM heat_summers');
```

Diese Zeile steht vor der Schleife.

Bei einer Live-Sammlung wäre Leeren fatal, weil die Vergangenheit nirgends sonst steht.

Dazuschreiben ohne Duplikate

Für eine Live-Sammlung, die über Wochen wächst.

- Eine `UNIQUE`-Regel auf Zeitpunkt und Ort
- Dazu `INSERT IGNORE` statt `INSERT`

Wenn etwas schiefgeht

```
$options = [  
    PDO::ATTR_ERRMODE          => PDO::ERRMODE_EXCEPTION,  
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,  
];
```

- Die erste Zeile sorgt dafür, dass Fehler laut abbrechen statt still zu scheitern
- Die zweite lässt PDO Zeilen als assoziative Arrays zurückgeben

Lest die Fehlermeldung von unten nach oben und sucht den Tabellen- oder Spaltennamen darin.

Die drei häufigsten Fehler sind Tippfehler im Spaltennamen, falsche Zugangsdaten und ein Fremdschlüssel ohne passende Stadt.

Zum Mitnehmen

Der Datenvertrag aus dem Transform ist bereits eine gute Grundlage für euer Datenmodell.

Nächster Schritt: das eigene Datenmodell auf Papier